

BME 133 Lab 4

Welcome!

To begin, create a copy of this file using File -> Save as... and name the new file "Lab_4_LastName_FirstName.ipynb" using your last and first name. Make sure to execute all the code cells in the notebook and that the outputs show in your report. Before submitting, remember to add a collaboration statement in this mark down indicating the contribution of each member. You may edit the template below:

All three of us worked on exercise 0 together.

- Dania Abuirbaleh - 018756387: Exercise 1
- Rohit Bhogal - 011221171: Exercise 2
- Neha Raj - 018968170: Exercise 3

This Lab will cover Classes, Inheritance and Modules.

Classes

Classes store data and functions. They allow you to:

- Model real-world problems, while hiding the details with abstraction
- Re-use code
- Encapsulate related data and behavior

Classes are defined using the *class* keyword. An example definition of a Car class is given below.

```
In [21]: class Car:
    def __init__(self, make, model, year, color):
        self.make = make
        self.model = model
        self.year = year
        self.color = color
        self.started = False
        self.speed = 0
        self.max_speed = 200

    def start(self):
        print("Starting the car... vroom, vroom!")
        self.started = True

    def stop(self):
        if self.speed > 40:
            print("Whoa, slow down first or you'll get whiplash!")
```

```

    else:
        print("Stopping the car...")
        self.started = False

    def accelerate(self, value):
        if not self.started:
            print("Car is not started!")
            return
        if self.speed + value <= self.max_speed:
            self.speed += value
        else:
            self.speed = self.max_speed
        print(f"Accelerating to {self.speed} mph...")

    def brake(self, value):
        if self.speed - value >= 0:
            self.speed -= value
        else:
            self.speed = 0
        print(f"Braking to {self.speed} mph...")

    def __str__(self):
        return f"This is a {self.color} {self.make} {self.model} ({self.year}

```

Classes contain data, also called *attributes* and functions, which in the context of classes are called *methods*. The Car class above contains the attributes make, model, year, color, started, speed and max speed. It has the methods start, stop, accelerate, brake. An *object* is an instance of the class with specific values for its attributes. To create an object of a class, you use the class name and a parenthesized list of values for its attributes as allowed by its `__init__()` method. See the first line in the code below for an example. Once you instantiate an object of a class such as the `mustang` object of the Car class below, you can access its other methods such as `start()`, `stop()`, `accelerate()`, and `brake()` using dot notation as shown in the next code block. The special `__str__` method is a method called when the object needs to be type cast to a string, such as in a print statement.

```

In [22]: mustang = Car('Ford', 'Mustang Boss 429', '1969', 'Red') # Instantiation: cr

# Examples of dot notation
mustang.start()
mustang.accelerate(35)
mustang.brake(10)
mustang.stop()

print(mustang) # Returns the string from the __str__ function

```

```

Starting the car... vroom, vroom!
Accelerating to 35 mph...
Braking to 25 mph...
Stopping the car...
This is a Red Ford Mustang Boss 429 (1969)

```

We can create multiple objects from the same class. We can create as many objects of our class as we need. To create another car object that is a Toyota Corolla 2025, we can follow a similar structure as before with the new arguments for the *constructor* `__init__` method of our car class as follows.

```
In [23]: newcar = Car('Toyota', 'Corolla', '2025', 'Black')
print(newcar)
```

This is a Black Toyota Corolla (2025)

Inheritance

Inheritance allows us to create a new *child* class extending an existing *parent* class by adding new attributes and methods. For instance, we can extend our Car class to a HybridCar class as follows.

```
In [24]: class HybridCar(Car):
def __init__(self, make, model, year, color, battery_voltage):
    super().__init__(make, model, year, color)
    self.battery_voltage = battery_voltage
def get_battery_voltage(self):
    return self.battery_voltage
def __str__(self):
    return f"This is a {self.color} {self.make} {self.model} ({self.year}
```

The super function calls the constructor of the parent class. We can instantiate a new object from the child class like that of the parent class:

```
In [25]: my_hybrid_car = HybridCar('Toyota', 'Camry', '2026', 'Black', 252)
print(my_hybrid_car)
```

This is a Black Toyota Camry (2026) and has a battery voltage of 252 volts

Notice how both Car and Hybrid Car had methods called `__str__`. This is a type of *method overriding* where a child class provides another implementation of a parent's method. When the method is called on an object of the child class, the implementation in the child class is used. Otherwise, the method in the parent class is used as the following code illustrates.

```
In [26]: my_hybrid_car.start()
```

Starting the car... vroom, vroom!

Built-in types

Recalling our discussions on data structures, you may infer now that lists have similar syntax as classes and the functions `append()`, `sort()`, etc we have been using are

methods of a list type. Consider the following code and think about what is happening in each line:

```
In [27]: my_shopping_list = list()
my_shopping_list.append("Milk")
my_shopping_list.append("Bread")
my_shopping_list.sort()
print(my_shopping_list)
```

```
['Bread', 'Milk']
```

In the first line, the list is constructed, i.e. the constructor method of the list is called. In the next two lines, the append method is called. Finally, the sort method is called. You can check the various methods available for a certain type by using the dot after the object. VSCode's IntelliSense feature will give you a list of suggestions available for that object.

Modules

In Python, a module or package is a collection of pre-defined functions and classes that can be used to save time. Python packages add a great deal of functionality. Standard modules are included with python, and don't have to be installed separately.

In later lessons you will learn how to manage packages that are installed on your device, but you will find that the Anaconda Distribution that you installed comes with many of the most useful packages included.

Here we see examples of some useful modules. For more information regarding Modules and Packages, I recommend:

General Module info

One useful module is the *time* module. Before using the time module's functions can be used you need to `import` the module. Reasoning:

- (1) It lets Python know which package you are talking about
- (2) It lets you specify how much of the package to load into memory

Functions and classes available in a module are accessed using a dot notation.

```
In [28]: # Example adapted from W3schools: https://www.w3schools.com/python/ref\_module\_time.py

import time
start = time.time()
print(f'Start time: {start}')
time.sleep(5) # Code wil take at least 5 seconds to run
end = time.time()
```

```
print(f'End time: {end}')
```

```
print(f'Elapsed: {end - start:.2f} seconds')
```

Start time: 1771651719.283791

End time: 1771651724.2890172

Elapsed: 5.01 seconds

The time() function in the time module returns the number of seconds since January 1, 1970. To get a more readable time, you can use the ctime function.

```
In [29]: time.ctime()
```

```
Out[29]: 'Fri Feb 20 21:28:44 2026'
```

Another useful module is the `math` module that provides access to common mathematical functions. For instance, to compute the number of bacteria after 5 rounds of binary division, we can take 2 to the power 5 using:

```
In [30]: import math
```

```
num_bacteria = math.exp2(5)
```

```
print(f'The number of bacteria after 5 rounds is: {num_bacteria}')
```

The number of bacteria after 5 rounds is: 32.0

The math module also contains some of the common constants. To compute the area of a Petri Dish of 50 mm radius, we can do so as follows:

```
In [31]: import math
```

```
petri_area = math.pi*math.pow(50, 2) # pow(50,2) = 50 to the power of 2
```

```
print(f'The area of a Petri Dish of 50 mm radius is: {petri_area}')
```

The area of a Petri Dish of 50 mm radius is: 7853.981633974483

To save time and space (memory) you can import individuals functions using the `from module_name import function_name` construct as shown in the code below. You can also import every function using the `*` wildcard. However, avoid using `from module_name import *` as this loads the whole module to memory, more memory = slower in some situations, and can cause errors.

```
In [32]: from math import pi, pow
```

```
petri_area = math.pi*math.pow(50, 2) # pow(50,2) = 50 to the power of 2
```

```
print(f'The area of a Petri Dish of 50 mm radius is: {petri_area}')
```

The area of a Petri Dish of 50 mm radius is: 7853.981633974483

Exercises

Excercise 0

(1 point) Create your own instance of the Class Car with attributes of your own car (or your close family/friend) and take it for a test drive. 🚗

```
In [9]: #Answer here

# Test Drive
# Instantiation: creating and specific instance of the Car class

# Use dot notation to access Car methods

# Return the string from the __str__ function

myCar = Car('Honda', "Civic", 2023, 'White')
print(myCar)
myCar.start()
myCar.accelerate(60)
myCar.brake(50)
myCar.stop()
```

```
This is a White Honda Civic (2023)
Starting the car... vroom, vroom!
Accelerating to 60 mph...
Braking to 10 mph...
Stopping the car...
```

Exercise 1

(4 points) Define a class called Implant. It should have a constructor that takes a material type (titanium, stainless steel, etc) as argument. When an object of the implant type is printed using a print function, it should output "This implant is made out of its_material_type". Test your definition by creating a titanium implant object.

```
In [12]: class Implant:
    def __init__(self, material_type):
        self.material_type = material_type

    def __str__(self):
        return f"This implant is made out of {self.material_type}"

steel_implant = Implant('stainless steel')

print(steel_implant)
```

```
This implant is made out of stainless steel
```

Excercise 2

(4 points) Create a child class of Implant called HipImplant. Its constructor should additionally take radius and height of the stem (in millimeters) of the implant, assuming for simplicity the stem is modelled as a cylinder. Override the __str__ function of the parent to additionally print these dimensions. Add a method called insertable_to_bone

that takes the radius and height of the cavity of the femur bone and returns true if the hip-implant-stem dimensions are less than or equal to the bone cavity dimensions, or false otherwise. Test your code with a hip implant that fits a bone and one that does not fit a bone. Add print statements as needed to show the functionality.

```
In [38]: class HipImplant(Implant):
    def __init__(self, material_type, stem_radius_mm, stem_height_mm):
        super().__init__(material_type)
        self.stem_radius_mm = stem_radius_mm
        self.stem_height_mm = stem_height_mm

    def __str__(self):
        return f"This implant is made out of {self.material_type} and it's s

# Refined Version
# -----
def insertable_to_bone(self, cavity_radius, cavity_height):
    radius_fits = self.stem_radius_mm <= cavity_radius
    height_fits = self.stem_height_mm <= cavity_height
    return radius_fits and height_fits

# Beginner Friendly Version V1
# -----
# def insertable_to_bone(self, cavity_radius, cavity_height):
#     if self.stem_radius_mm <= cavity_radius and self.stem_height_mm <=
#         return True
#     else:
#         return False

# Beginner Friendly Version V2
# def insertable_to_bone(self, cavity_radius, cavity_height):
#     radius_fits = self.stem_radius_mm <= cavity_radius
#     height_fits = self.stem_height_mm <= cavity_height

#     if radius_fits and height_fits:
#         return True
#     return False

steel_hip_implant = HipImplant('stainless steel', 50, 100)

print(steel_hip_implant)

print(steel_hip_implant.insertable_to_bone(60, 200)) # True
print(steel_hip_implant.insertable_to_bone(40, 90)) # False
print(steel_hip_implant.insertable_to_bone(60, 90)) # False
```

This implant is made out of stainless steel and it's stem radius is 50 and it's stem height is 100

True
False
False

Excercise 3

(1 points) Use functions in the math module to compute the pH for an $[H^+]$ concentration of 2×10^{-7} . The pH is given by $pH = -\log_{10}[H^+]$.

```
In [33]: import math

h_concentration = 2e-7
pH = -math.log10(h_concentration)

print(pH)
```

6.698970004336019